One Dimension Optimization:
Bisection, Newton, Secant (Lec7.pdf)

Ioannis Akrotirianakis

Todays topics & Logistics

- One Dimensional Optimization
  - Motivation
  - Bisection method
  - Newton method
  - Secant method

Motivation for 1 dimensional optimization ($n = 1$)

- **Why do we need to know** 1 dimensional optimization ?? ($n = 1$)
  - It is used as a **subroutine** (function) in **multi-dimensional** optimization (when $n > 1$)
  - Newton method **generalizes** to many dimensions *very* naturally
  - **Challenges and difficulties** of optimization methods for 1 dim.opt. help us understand similar challenges in many dimensions
- But in 1 dimension we can **simply plot** the function and find its optimum
  - Plotting requires **too many function evaluations** which are computationally expensive!
  - **Optimization algorithms find the minimum with the least number of function evaluations**

Bisection method

- It is one of the **simplest** optimization methods → Very similar to the process of locating a book in a library
- **Intuition**:
  - We know that the minimum of a continuously differentiable function $f: R \to R$ is somewhere in the interval $[a_0, b_0]$
  - If the function is convex what would happen to its **derivative** in the interval??

Bisection method

- It is one of the **simplest** optimization methods → Very similar to the process of locating a book in a library
- **Intuition**:
  - We know that the minimum of a continuously differentiable function $f: R \to R$ is somewhere in the interval $[a_0, b_0]$
  - If the function is convex what would happen to its **derivative** in the interval??
    - There will be a point $x^* \in [a_0, b_0]$ where $f'(x^*) = 0$, and
    - For all $x \in [a_o, x^*)$ we have $f'(x) < 0$ and f is decreasing, and
    - For all $x \in (x^*, b_0]$ we have $f'(x^*) > 0$

$\boldsymbol{f'(x)}$ **will change sign**

Bisection method

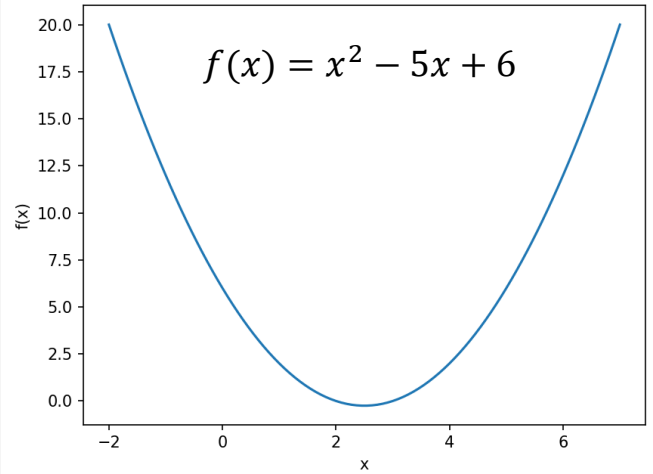
Pseudocode:

for $k = 0{:}\,N$
  if $f'(\frac{a_k+b_k}{2}) = 0$
    you are done;
  elseif $f'(\frac{a_k+b_k}{2}) > 0$
    $a_{k+1} = a_k;$
    $b_{k+1} = (a_k + b_k)/2;$
  else (if $f'(\frac{a_k+b_k}{2})<0$)
    $b_{k+1} = b_k;$
    $a_{k+1} = (a_k + b_k)/2;$
  end
end

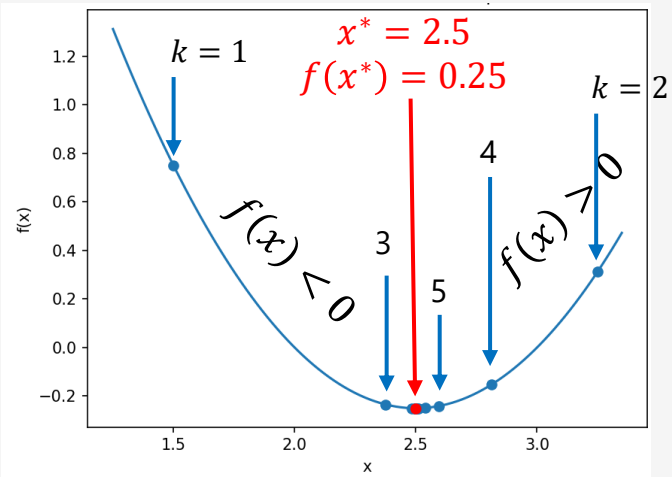
Example: find the minimum of $f(x) = x^2 - 5x + 6$ in the interval $[-2,5]$

$\boldsymbol{x_k}$ = [1.5, 3.25, 2.375, 2.8125, 2.59375, 2.484375, 2.539062, 2.511718, 2.498046, 2.504882, 2.501464, 2.499755, 2.500610, 2.5001]

$\boldsymbol{f(x_k)}$ =[0.75, 0.3125, -0.23437, -0.15234, -0.24121, -0.24975, -0.24847, -0.24986, -0.24999, -0.24997, -0.2499978, -0.24999994, -0.2499996, -0.24999996]



Zoomed in [1 , 3.5]



Bisection method

- **Remarks**:
  - It is an **iterative algorithm** → at every iteration we are closer to the optimal solution
  - The length of the solution interval reduces by **½** at every iteration
    - The rate of convergence is linear (discussed later…)
  - If our tolerance is ϵ then the optimal solution will be reached after $N = \log_2 \frac{|a_0 - b_0|}{\epsilon}$ iterations
  - It is a **robust algorithm** (it will converge) but it will be **slow** *(even when it is close to the optimal solution)*
  - We can use the Bisection algorithm for **root finding**, in fact this is what we implicitly did, i.e., find the root of the equation: $f'(x) = 0$

Newton method

- Uses an iterative formula to generate a sequence of points that approach the optimal solution of the optimization problem $\min_{x \in R} f(x)$

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)} \quad \text{and} \quad \lim_{k \to \infty} x_k = x^*$$

- **Motivations:**
  - #1: local quadratic approximation of f
  - #2: method of tangents – solve the equation: $f'(x) = 0$ (FONC)

Newton method – local quadratic approximation

- **2nd order Taylor approximation around x_k**

$$q(x) = f(x_k) + f'(x_k)(x - x_k) + \frac{1}{2} f''(x_k)(x - x_k)^2$$

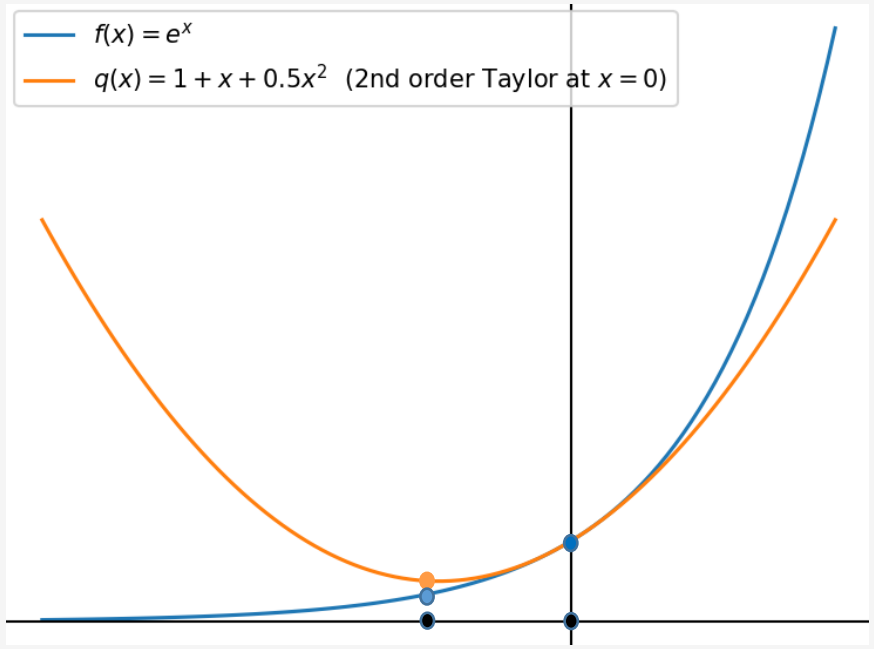
  - q, q', q'' **match** f, f', f'' at x_k

- $q(x)$ is a **quadratic function**: $q(x) = ax^2 + bx + c$

  - where $a = \frac{f''(x_k)}{2}$ and $b = f'(x_k) - f''(x_k)x_k$ are known.

- Using **FONC**: $q'(x) = 0 \Rightarrow 2ax + b = 0 \Rightarrow x_{k+1} = -\frac{b}{2a}$

  - Hence $x_{k+1} = -\frac{f'(x_k) - f''(x_k)x_k}{f''(x_k)} \Rightarrow x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$



$x_{k+1} = -1$ $x_k = 0$

Newton method – local quadratic approximation

- **Step 0:** Start from an initial point x_0 (random, or given) and set $k = 0$
- **Step 1:** Evaluate f, f', f'' at the current point x_k
- **Step 2:** Compute next iterate $x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$ *(you optimize the quadratic approx. @ x_k)*
- **Step 4:** If $|x_{k+1} - x_k| < \epsilon$ then set $x^* = x_{k+1}$ and STOP;
  - Else set $x_k = x_{k+1}$ and set $k = k + 1$ and GOTO Step 1

Newton method – method of tangents/finding roots of $f'(x) = 0$

- In a way, we find the **roots of $f'(x) = 0$** (i.e., FONC)

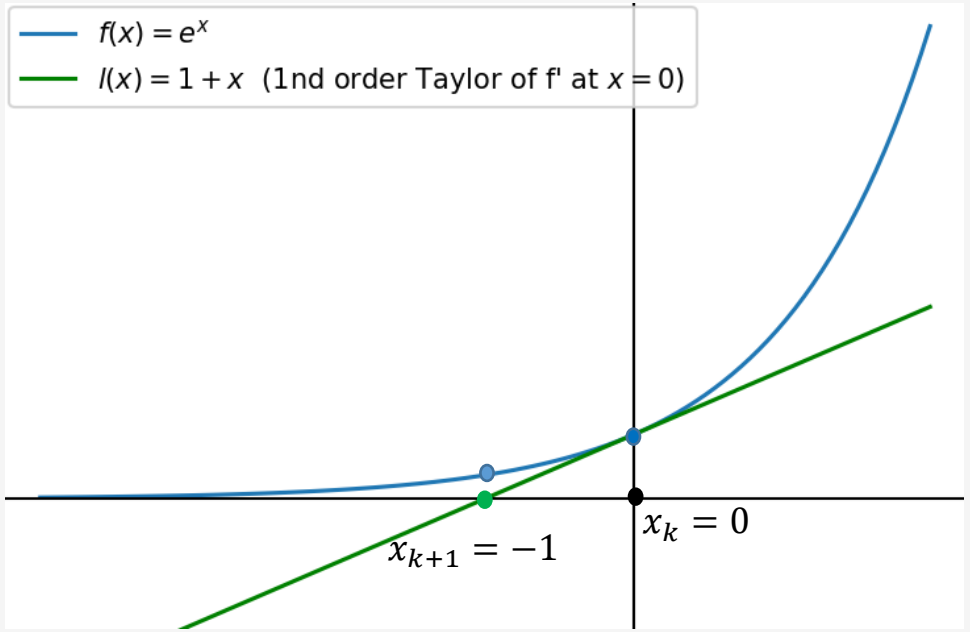
- If we define: $\boldsymbol{g(x) = f'(x)}$, and approximate g by its **1st order Taylor expansion** (a line):

  - $l(x) = g(x_k) + g'(x_k)(x - x_k)$

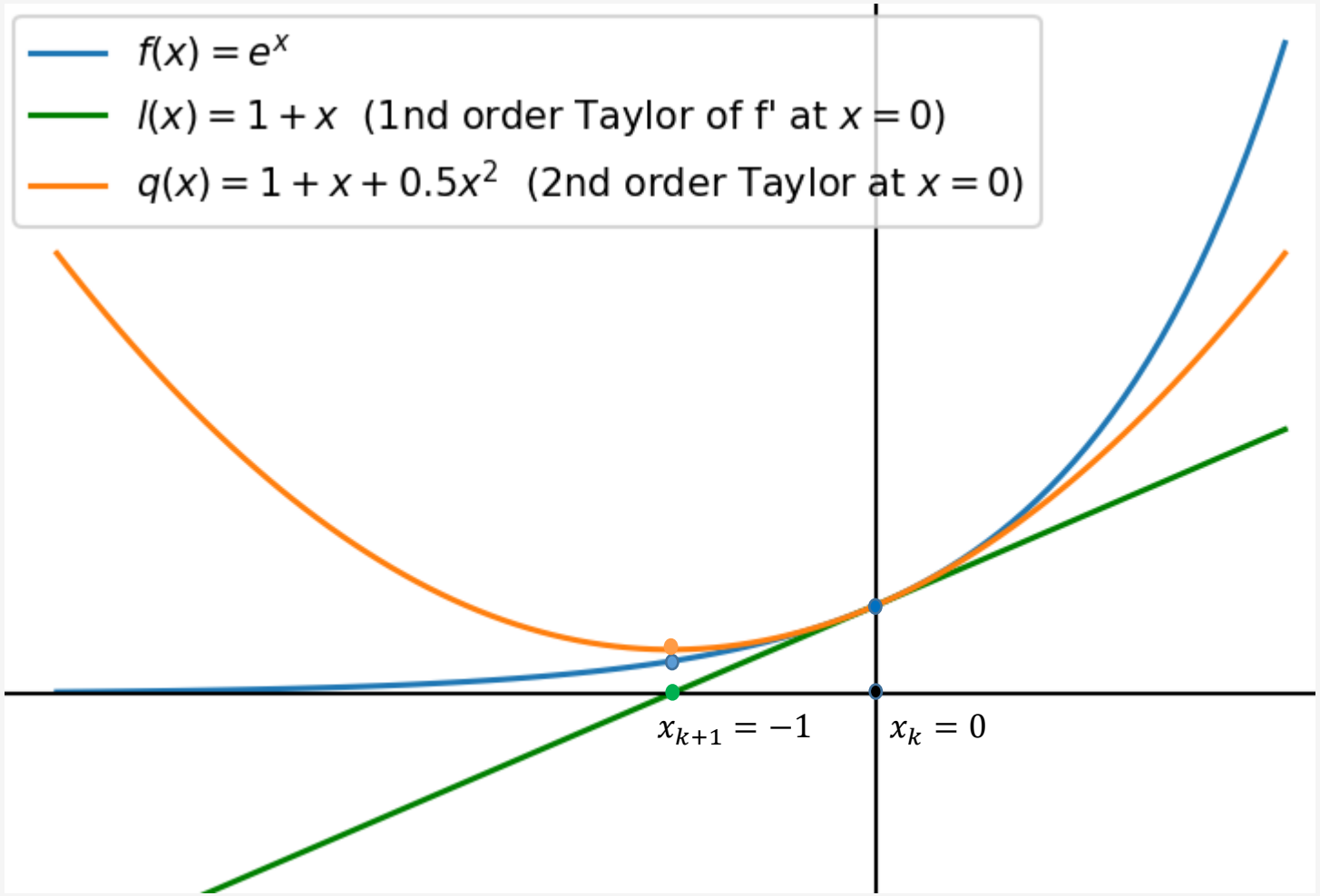
- We take as next iterate the point $\boldsymbol{x_{k+1}}$ that makes $\boldsymbol{l(x) = 0}$

  - $l(x_{k+1}) = 0 \Rightarrow g(x_k) + g'(x_k)(x_{k+1} - x_k) = 0 \Rightarrow$

$$\Rightarrow x_{k+1} = x_k - \frac{g(x_k)}{g'(x_k)} \quad \Rightarrow \quad x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$$



Newton method – both motivations in one plot



The good and the bad of Newton method (*but no ugly*)

The good:

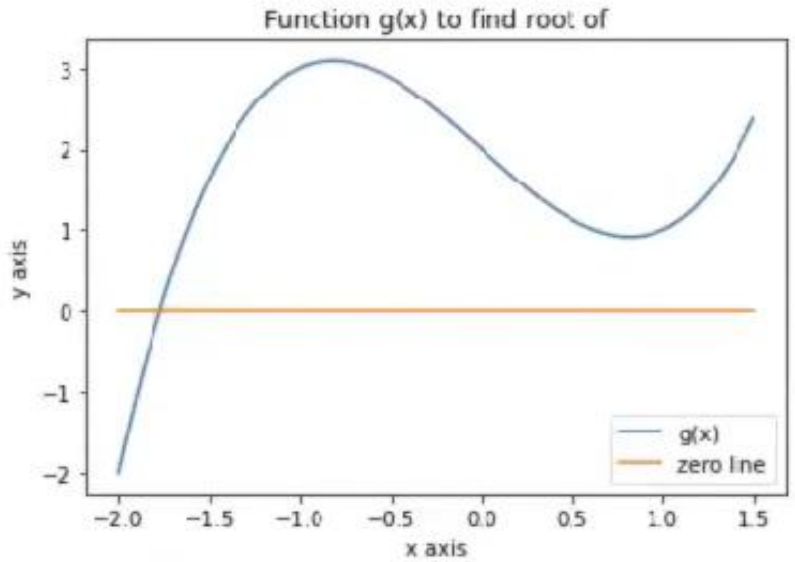
- It converges **very fast**, when started **close to a local optimum**
  - **Quadratic convergence** → number of correct digits **doubles** at every iteration when close to x^*

The bad:

- Convergence is **sensitive to starting points** → **Newton fractals**
- The iterations may **diverge**
- Convergence may be **slow** if $f''(x^*) = 0$

Newton is sensitive to the initial point

```
g_bad1 = lambda x: x**3 - 2*x +2
dg_bad1=lambda x: 3*x**2 - 2
plot_g(g_bad1,-2,1.5)
```




```
n_bad1 = newton(g_bad1,dg_bad1,0,10)
```

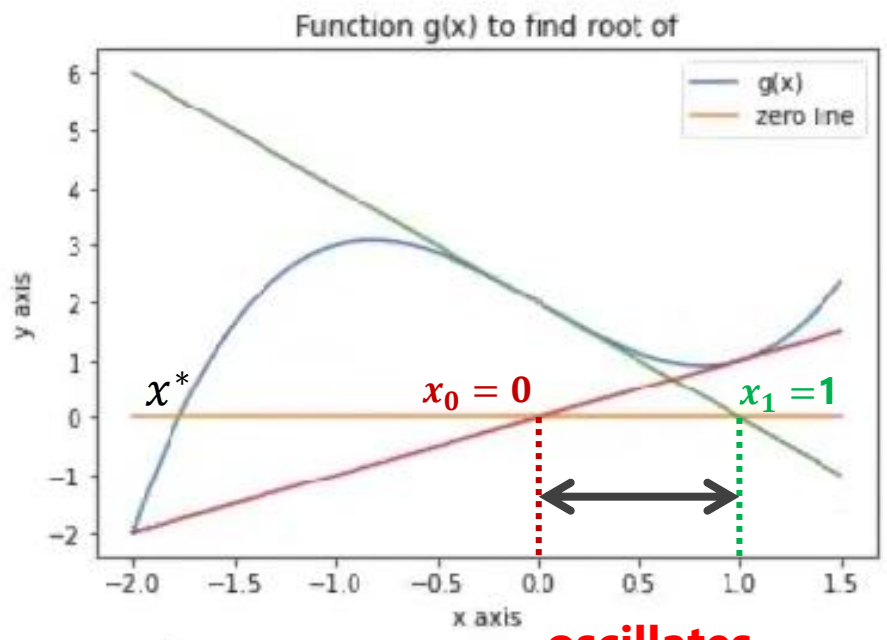
$x_0 = 0$

```
iteration 01 0
iteration 02 1.0
iteration 03 0.0
iteration 04 1.0
iteration 05 0.0
iteration 06 1.0
iteration 07 0.0
iteration 08 1.0
iteration 09 0.0
iteration 10 1.0
```

Newton can **oscillate for ever**;
And never reach the solution…

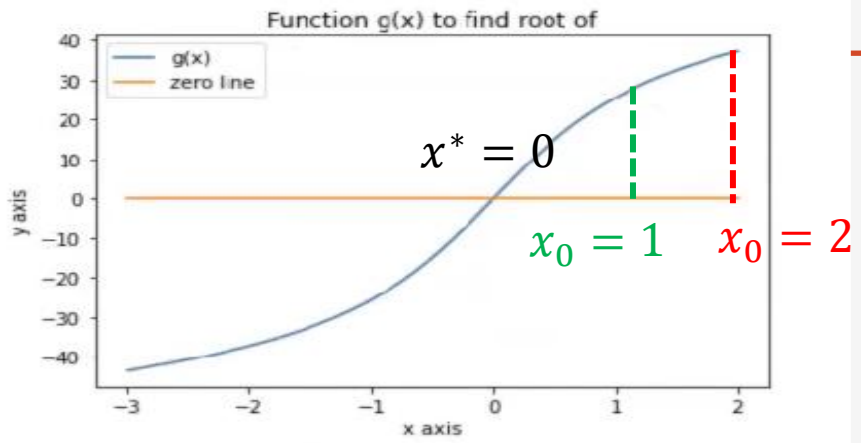
$\mathbf{f}(x) = x^3 - 2x + 2 = 0$

```
[13] plot_g(g_bad1,-2,1.5, dg_bad1,0,1)
```



oscillates

```
[10] g_bad2 = lambda x: 30* np.arctan(x) + 2*x
     dg_bad2=lambda x: 30/(1+(x)**2) + 2
     plot_g(g_bad2,-3,2)
```



Newton is sensitive to the initial point

Newton can **diverge** if x_0 is far from the solution

$$f(x) = 30\arctan(x) + 2x = 0$$

```
n_bad2 = newton(g_bad2,dg_bad2,1,10)
```

```
iteration 01 1
iteration 02 -0.5036438177602027
iteration 03 0.07499276542612121
iteration 04 -0.000263207288816375
iteration 05 1.1396559043697074e-11
iteration 06 0.0
iteration 07 0.0
iteration 08 0.0
iteration 09 0.0
iteration 10 0.0
```

$x_0 = 1$

converges

```
n_bad2 = newton(g_bad2,dg_bad2,2,10)
```

```
iteration 01 2
iteration 02 -2.6518076917278393
iteration 03 4.60346823295529
iteration 04 -10.287948780125504
iteration 05 18.120156218978632
iteration 06 -20.955261867862188
iteration 07 21.4030471541725
iteration 08 -21.461106011682343
iteration 09 21.46842428563779
iteration 10 -21.46934342619276
```

$x_0 = 2$

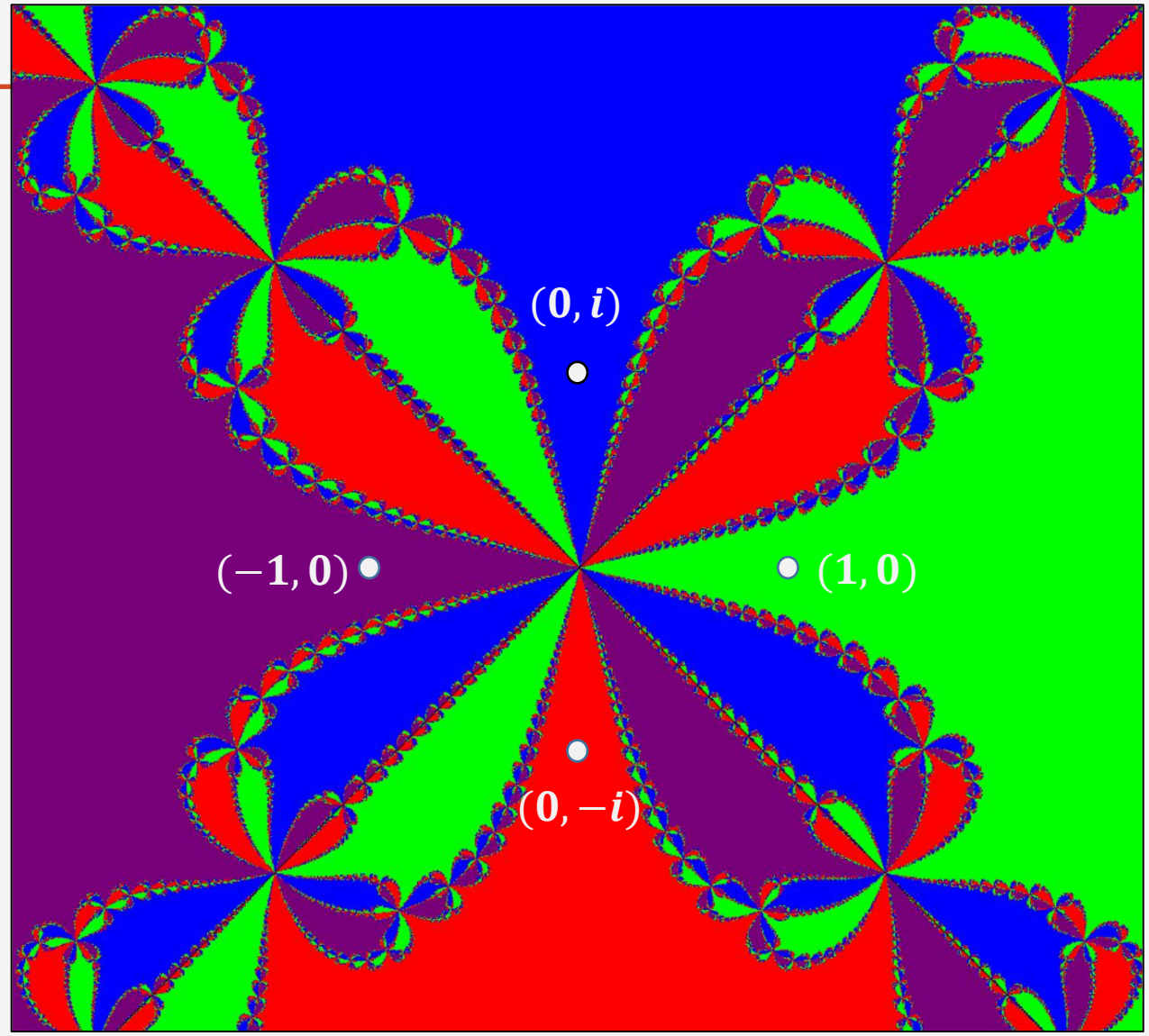
diverges

Newton fractals

Newton fractals created by the **regions** of **attraction** to different solution for the equation:

$$z^4 - 1 = 0$$

In the complex space, there are 4 solutions:
$(1, 0)$, $(-1, 0)$, $(0, i)$, $(0, -i)$



Note: in general, the roots of the equation: $z^n - 1 = 0$ in the complex space are given by: $z_m = e^{i\theta_m}$, where $\theta_m = \frac{2\pi m}{n}$ and $m = 0,1, \dots, n-1$. Also note that i is the imaginary number in the complex space(here we have: $i = \sqrt{-1}$, the imaginary number) → see next slide for more details and an example

Newton fractals

Note that in general, the roots of the equation: $\boldsymbol{z^n - 1 = 0}$ in the **complex space** are given by:

$$z_m = e^{i\theta_m}$$

where $\theta_m = \frac{2\pi m}{n}$ and $m = 0,1,\dots,n-1$.

So, in the complex space the equation $z^n - 1 = 0$ has $\boldsymbol{n}$ **distinct roots** (as expected).

Also note that $\boldsymbol{i}$ is the imaginary number in the complex space ($\boldsymbol{i} = \sqrt{-\mathbf{1}}$).

Recall that **Euler's formula** tells us that:

$$e^{i\theta_m} = \cos(\theta_m) + i\,sin(\theta_m)$$

Hence, using Euler's formula we can determine all n different complex roots of the equation

$$z^n - 1 = 0.$$

Simple example: consider the equation: $z^4 - 1 = 0$

Using Euler's formula, we can derive all 4 roots.

First, we calculate the θ_ms, for $m = 0,1,2,3$:

$$m = 0 \Rightarrow \theta_0 = \frac{2\pi 0}{4} = 0,$$

$$m = 1 \Rightarrow \theta_1 = \frac{2\pi 1}{4} = \pi/2$$

$$m = 2 \Rightarrow \theta_2 = \frac{2\pi 2}{4} = \pi$$

$$m = 3 \Rightarrow \theta_3 = \frac{2\pi 3}{4} = 3\pi/2$$

Then we can derive the roots:

$$\mathrm{m} = 0 \Rightarrow z_0 = e^{i\theta_0} = \cos(\theta_0) + i\,sin(\theta_0) = 1$$
$$\mathrm{m} = 1 \Rightarrow z_1 = e^{i\theta_1} = \cos(\theta_1) + i\,sin(\theta_1) = i$$
$$\mathrm{m} = 2 \Rightarrow z_2 = e^{i\theta_2} = \cos(\theta_2) + i\,sin(\theta_2) = -1$$
$$\mathrm{m} = 3 \Rightarrow z_3 = e^{i\theta_3} = \cos(\theta_3) + i\,sin(\theta_3) = -i$$

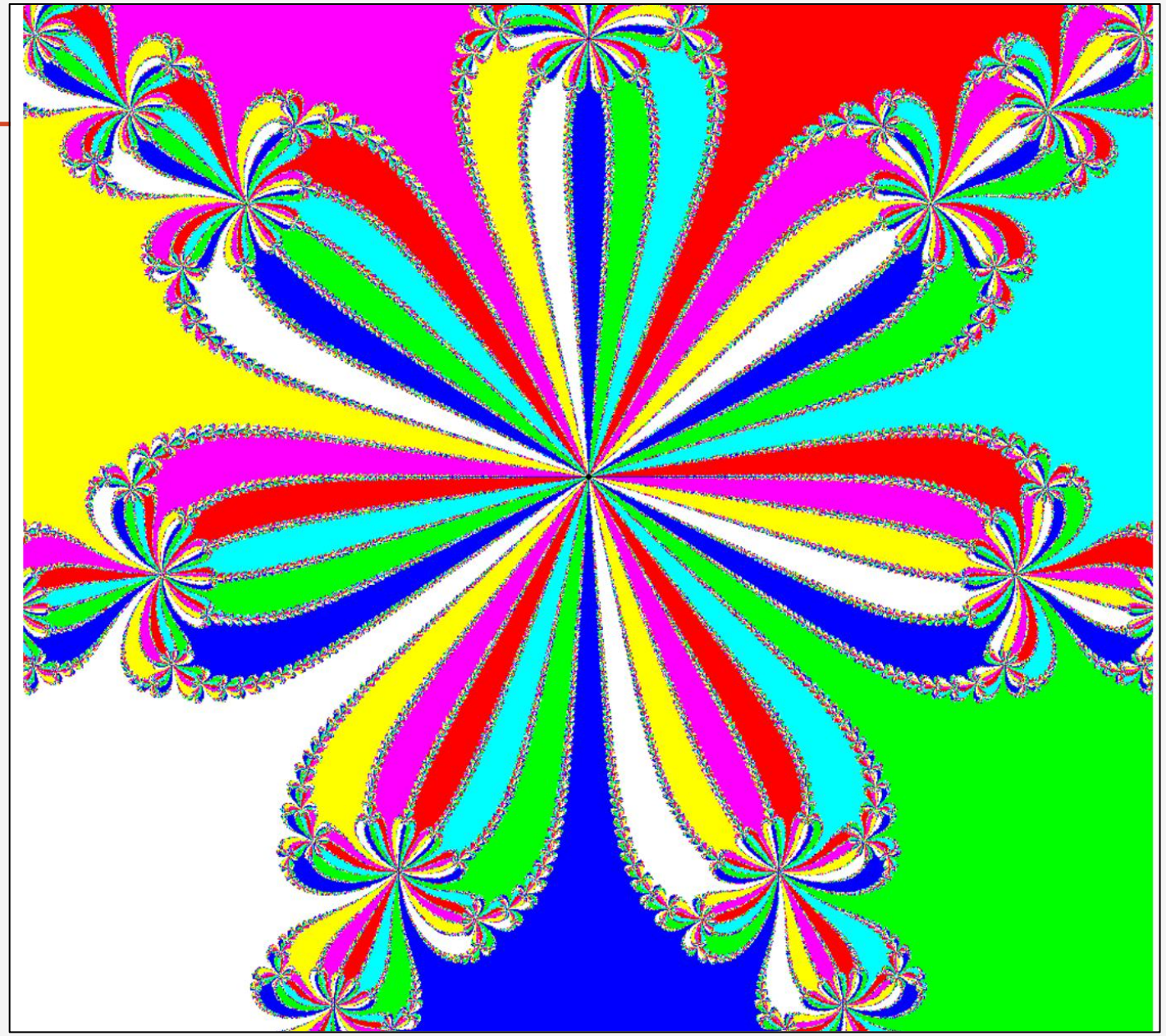
You can try to solve higher dimensional equations of the same form and easily calculate the roots following the same process.

Newton fractals

Newton fractals created by the **regions** of **attraction** to different solution for the equation:

$$z^7 - 1 = 0$$

In the complex space there are 7 distinct solutions with many distinct regions of attractions for each solution.



Secant methods

- Newton method requires the computation of **2nd derivatives:** f''
  - It is computational **expensive** task
- The **Secant method** approximates f'' using the **finite difference** on f' (it is cheaper computationally)

$$f''(x_k) \approx \frac{f'(x_k) - f'(x_{k-1})}{x_k - x_{k-1}}$$

We need two points:
x_k and x_{k-1}

  - Substituting the finite difference approximation in the Newton iteration

do the algebra

$$x_{k+1} = x_k - \frac{x_k - x_{k-1}}{f'(x_k) - f'(x_{k-1})} f'(x_k) \quad \Rightarrow \quad x_{k+1} = \frac{f'(x_k)x_{k-1} - f'(x_{k-1})x_k}{f'(x_k) - f'(x_{k-1})}$$

It needs more iterations than the Newton method but less iterations than Bisection method